

COM S // CPR E // MATH 4240

Introduction to High-Performance Computing

Instructor: Songting Luo

Lecture 13: Matrix Struct and Finding Eigenvalues

Outline

1. Matrix Struct
2. Finding Eigenvalues

Matrix Struct

Matrix Struct: header file (struct + macros)

\$ISUHPC/lectures/lecture13/codes/matrix/matrix.h:

```
1  #ifndef __MATRIX_H__
2  #define __MATRIX_H__
3
4  typedef struct matrix matrix;
5  struct matrix
6  { int rows; int cols; double* val; };
7
8  typedef struct vector vector;
9  struct vector
10 { int size; double* val; };
11
12 // Shortcut evaluate functions
13 #define mget(mat,i,j) mat.val[(i-1)*mat.cols+(j-1)]
14 #define mgetp(mat,i,j) mat->val[(i-1)*mat->cols+(j-1)]
15 #define vget(vec,i) vec.val[(i-1)]
16 #define vgetp(vec,i) vec->val[(i-1)]
17 #define print_matrix(mat) print_matrix_full(mat,#mat);
18 #define print_vector(vec) print_vector_full(vec,#vec);
19 #define print_scalar(z) print_scalar_full(z,#z);
```

Matrix Struct: header file (declarations)

\$ISUHPC/lectures/lecture13/codes/matrix/matrix.h:

```
1 // Function declarations
2 matrix new_matrix(const int rows, const int cols);
3 void print_matrix_full(const matrix* mat, char* varname);
4 matrix matrix_add(const matrix* A, const matrix* B);
5 matrix matrix_sub(const matrix* A, const matrix* B);
6 matrix matrix_mult(const matrix* A, const matrix* B);
7 matrix matrix_dot_mult(const matrix* A, const matrix* B);
8 //
9 vector new_vector(const int size);
10 void print_vector_full(const vector* vec, char* varname);
11 vector vector_add(const vector* x, const vector* y);
12 vector vector_sub(const vector* x, const vector* y);
13 double vector_dot_mult(const vector* x, const vector* y);
14 //
15 void print_scalar_full(const double* z, char* varname);
16 //
17 vector matrix_vector_mult(const matrix* A, const vector* x);
18 //
19 vector solve(const matrix* A, const vector* b);
20
21 #endif
```

Matrix Struct: source file (create a new matrix)

\$ISUHPC/lectures/lecture13/codes/matrix/matrix.c:

```
1  #include <stdlib.h>
2  #include <stdio.h>
3  #include <assert.h>
4  #include <math.h>
5  #include "matrix.h"
6
7  matrix new_matrix(const int rows, const int cols)
8  {
9      matrix mat;
10     mat.rows = rows;
11     mat.cols = cols;
12     assert(rows>0);
13     assert(cols>0);
14     mat.val = (double*)malloc(sizeof(double)*rows*cols);
15
16     for (int i=0; i<(rows*cols); i++)
17     {  mat.val[i] = 0.0;  }
18
19     return mat;
20 }
```

Matrix Struct: source file (print matrix)

\$ISUHPC/lectures/lecture13/codes/matrix/matrix.c:

```
1  void print_matrix_full(const matrix* mat, char* varname)
2  {
3      assert(mat->rows>0); assert(mat->cols>0);
4      printf("\n %.100s =\n", &varname[1] );
5      for(int i=1; i<=mat->rows; i++ )
6      {
7          printf("  |  ");
8          for(int j=1; j<=mat->cols; j++)
9          {
10             printf("%10.3e",mgetp(mat,i,j));
11             if (j<mat->cols) {printf(", ");}
12             else {printf(" ");}
13         }
14         printf("|\\n");
15     }
16     printf("\\n");
17 }
```

```
1  #define print_matrix(mat) print_matrix_full(mat,#mat);
```

```
1  #define mgetp(mat,i,j) mat->val[(i-1)*mat->cols+(j-1)]
```

Matrix Struct: source file (add two matrices)

\$ISUHPC/lectures/lecture13/codes/matrix/matrix.c:

```
1  matrix matrix_add(const matrix* A, const matrix* B)
2  {
3      const int rows = A->rows;
4      const int cols = A->cols;
5      assert(rows==B->rows);
6      assert(cols==B->cols);
7      matrix C = new_matrix(rows,cols);
8
9      for (int i=1; i<=rows; i++)
10         for (int j=1; j<=cols; j++)
11             {
12                 mget(C,i,j) = mgetp(A,i,j)+mgetp(B,i,j);
13             }
14
15     return C;
16 }
```

```
1  #define mget(mat,i,j) mat.val[(i-1)*mat.cols+(j-1)]
2  #define mgetp(mat,i,j) mat->val[(i-1)*mat->cols+(j-1)]
```

Matrix Struct: source file (matrix-matrix product)

\$ISUHPC/lectures/lecture13/codes/matrix/matrix.c:

```
1  matrix matrix_mult(const matrix* A, const matrix* B)
2  {
3      const int rowsA = A->rows; const int colsA = A->cols;
4      const int rowsB = B->rows; const int colsB = B->cols;
5      assert(colsA==rowsB);
6      matrix C = new_matrix(rowsA,colsB);
7
8      for (int i=1; i<=rowsA; i++)
9          for (int j=1; j<=colsB; j++)
10             for (int k=1; k<=colsA; k++)
11                 {
12                     mget(C,i,j) += mgetp(A,i,k)*mgetp(B,k,j);
13                 }
14
15     return C;
16 }
```

```
1  #define mget(mat,i,j) mat.val[(i-1)*mat.cols+(j-1)]
2  #define mgetp(mat,i,j) mat->val[(i-1)*mat->cols+(j-1)]
```

Matrix Struct: source file (element-by-element product)

\$ISUHPC/lectures/lecture13/codes/matrix/matrix.c:

```
1  matrix matrix_dot_mult(const matrix* A, const matrix* B)
2  {
3      const int rows = A->rows;
4      const int cols = A->cols;
5      assert(rows==B->rows);
6      assert(cols==B->cols);
7      matrix C = new_matrix(rows,cols);
8
9      for (int i=1; i<=rows; i++)
10         for (int j=1; j<=cols; j++)
11             {
12                 mget(C,i,j) = mgetp(A,i,j)*mgetp(B,i,j);
13             }
14
15     return C;
16 }
```

```
1  #define mget(mat,i,j) mat.val[(i-1)*mat.cols+(j-1)]
2  #define mgetp(mat,i,j) mat->val[(i-1)*mat->cols+(j-1)]
```

Matrix Struct: source file (create vector)

\$ISUHPC/lectures/lecture13/codes/matrix/matrix.c:

```
1  vector new_vector(const int size)
2  {
3      vector vec;
4      vec.size = size;
5      assert(size>0);
6      vec.val = (double*)malloc(sizeof(double)*size);
7
8      for (int i=0; i<(size); i++)
9      {  vec.val[i] = 0.0;  }
10
11     return vec;
12 }
```

Matrix Struct: source file (print vector)

\$ISUHPC/lectures/lecture13/codes/matrix/matrix.c:

```
1 void print_vector_full(const vector* vec, char* varname)
2 {
3     assert(vec->size>0);
4     printf("\n");
5     printf(" %.100s =\n", &varname[1] );
6     printf(" | ");
7     for(int i=1; i<=vec->size; i++ )
8     {
9         printf("%10.3e",vgetp(vec,i));
10        if (i<vec->size) {printf(", ");}
11    }
12    printf(" |^T\n\n");
13 }
```

```
1 #define print_vector(vec) print_vector_full(vec,#vec);
```

Matrix Struct: source file (add two vectors)

\$ISUHPC/lectures/lecture13/codes/matrix/matrix.c:

```
1  vector vector_add(const vector* x, const vector* y)
2  {
3      const int size = x->size;
4      assert(size==y->size);
5      vector z = new_vector(size);
6
7      for (int i=1; i<=size; i++)
8      {
9          vget(z,i) = vgetp(x,i)+vgetp(y,i);
10     }
11
12     return z;
13 }
```

```
1  #define vget(vec,i) vec.val[(i-1)]
2  #define vgetp(vec,i) vec->val[(i-1)]
```

Matrix Struct: source file (dot product of two vectors)

\$ISUHPC/lectures/lecture13/codes/matrix/matrix.c:

```
1  double vector_dot_mult(const vector* x, const vector* y)
2  {
3      const int size = x->size; assert(size==y->size);
4
5      double z = 0.0;
6      for (int i=1; i<=size; i++)
7          { z += vgetp(x,i)*vgetp(y,i); }
8
9      return z;
10 }
11
12 void print_scalar_full(const double* z, char* varname)
13 {
14     printf("\n %.100s =\n", &varname[1] );
15     printf("      %10.3e \n\n", *z);
16 }
```

```
1  #define vget(vec,i) vec.val[(i-1)]
```

```
1  #define print_scalar(z) print_scalar_full(z,#z);
```

Matrix Struct: source file (matrix-vector multiply)

\$ISUHPC/lectures/lecture13/codes/matrix/matrix.c:

```
1  vector matrix_vector_mult(const matrix* A, const vector* x)
2  {
3      const int rows = A->rows; const int cols = A->cols;
4      const int size = x->size;
5      assert(cols==size);
6      vector Ax = new_vector(rows);
7
8      for (int i=1; i<=rows; i++)
9      {
10         double tmp = 0.0;
11         for (int j=1; j<=size; j++)
12             { tmp += mgetp(A,i,j)*vgetp(x,j); }
13         vget(Ax,i) = tmp;
14     }
15
16     return Ax;
17 }
```

```
1  #define mgetp(mat,i,j) mat->val[(i-1)*mat->cols+(j-1)]
2  #define vget(vec,i) vec->val[(i-1)]
3  #define vgetp(vec,i) vec->val[(i-1)]
```

Matrix Struct: source file (Gaussian elimination with partial pivoting)

$$A = \begin{bmatrix} 2 & 1 & 1 & 0 \\ 4 & 3 & 3 & 1 \\ 8 & 7 & 9 & 5 \\ 6 & 7 & 9 & 8 \end{bmatrix}$$

$$\rightarrow \begin{bmatrix} 8 & 7 & 9 & 5 \\ 4 & 3 & 3 & 1 \\ 2 & 1 & 1 & 0 \\ 6 & 7 & 9 & 8 \end{bmatrix} \rightarrow \begin{bmatrix} 8 & 7 & 9 & 5 \\ -1/2 & -3/2 & -3/2 & \\ -3/4 & -5/4 & -5/4 & \\ 7/4 & 9/4 & 17/4 & \end{bmatrix}$$

$$\rightarrow \begin{bmatrix} 8 & 7 & 9 & 5 \\ & 7/4 & 9/4 & 17/4 \\ & -3/4 & -5/4 & -5/4 \\ & -1/2 & -3/2 & -3/2 \end{bmatrix} \rightarrow \begin{bmatrix} 8 & 7 & 9 & 5 \\ & 7/4 & 9/4 & 17/4 \\ & & -2/7 & 4/7 \\ & & -6/7 & -2/7 \end{bmatrix}$$

$$\rightarrow \begin{bmatrix} 8 & 7 & 9 & 5 \\ & 7/4 & 9/4 & 17/4 \\ & & -6/7 & -2/7 \\ & & -2/7 & 4/7 \end{bmatrix} \rightarrow \begin{bmatrix} 8 & 7 & 9 & 5 \\ & 7/4 & 9/4 & 17/4 \\ & & -6/7 & -2/7 \\ & & & 2/3 \end{bmatrix}$$

Matrix Struct: source file (Gaussian elimination with partial pivoting)

\$ISUHPC/lectures/lecture13/codes/matrix/matrix.c:

```
1  vector solve(const matrix* A, const vector* b)
2  {
3      const int rows = A->rows; const int cols = A->cols;
4      const int size = b->size;
5      assert(rows==cols); assert(rows==size);
6
7      vector x = new_vector(rows);
8
9      for (int i=1; i<=(size-1); i++) // LOOP OVER EACH COLUMN
10     {
11         // Select largest pivot in current column
12         int p=i; double maxA = -100.0e0;
13         for (int j=i; j<=size; j++)
14         {
15             double tmp = fabs(mgetp(A,j,i));
16             if ( tmp > maxA ){ p = j; maxA = tmp; }
17         }
18
19         // See if matrix is singular
20         if (maxA <= 1.0e-14)
21             { printf(" Cannot invert system\n"); exit(1); }
```

Matrix Struct: source file (Gaussian elimination with partial pivoting)

\$ISUHPC/lectures/lecture13/codes/matrix/matrix.c:

```
1 // Pivot (aka interchange rows)
2 if (p!=i)
3 {
4     for (int j=1; j<=size; j++)
5     {
6         double tmp1 = mgetp(A,i,j);
7         mgetp(A,i,j) = mgetp(A,p,j); mgetp(A,p,j) = tmp1;
8     }
9
10    double tmp2 = vgetp(b,i);
11    vgetp(b,i) = vgetp(b,p); vgetp(b,p) = tmp2;
12 }
13
14 // Eliminate below diagonal
15 for (int j=(i+1); j<=size; j++)
16 {
17     double dm = mgetp(A,j,i)/mgetp(A,i,i);
18     for (int k=1; k<=size; k++)
19     { mgetp(A,j,k) = mgetp(A,j,k) - dm*mgetp(A,i,k); }
20     vgetp(b,j) = vgetp(b,j) - dm*vgetp(b,i);
21 }
22 }
```

Matrix Struct: source file (Gaussian elimination with partial pivoting)

\$ISUHPC/lectures/lecture13/codes/matrix/matrix.c:

```
1  // Backward substitution
2  vget(x,size) = vgetp(b,size)/mgetp(A,size,size);
3  for (int j=1; j<=(size-1); j++)
4  {
5      double sum = 0.0e0;
6
7      for (int k=(size-j+1); k<=size; k++)
8          { sum = sum + mgetp(A,size-j,k)*vget(x,k); }
9
10     vget(x,size-j) = (vgetp(b,size-j) - sum)
11         /mgetp(A,size-j,size-j);
12 }
13
14 return x;
15 }
```

Matrix Struct: Makefile

\$ISUHPC/lectures/lecture13/codes/matrix/Makfile:

```
1  # $ISUHPC/lectures/lecture12/codes/matrix/Makefile
2  CC = gcc
3  FFLAGS = -O3 -Wall -std=c99
4  LFFLAG =
5  OBJECTS = main.o \
6  matrix.o
7
8  .PHONY: clean
9
10 main.exe: $(OBJECTS)
11     $(CC) $(LFLAGS) $(OBJECTS) -o main.exe
12
13 %.o: %.c
14     $(CC) $(FFLAGS) -c $<
15
16 clean:
17     rm -f $(OBJECTS) main.exe
```

Matrix Struct: main function

\$ISUHPC/lectures/lecture13/codes/matrix/main.c:

```
1  #include <stdlib.h>
2  #include <stdio.h>
3  #include "matrix.h"
4
5  int main()
6  {
7      // Matrices
8      matrix A = new_matrix(5,5);
9      matrix B = new_matrix(5,5);
10
11     for(int i=1; i<=5; i++ )
12         for (int j=1; j<=5; j++ )
13             {
14                 mget(A,i,j) = -1.0*(i==j)
15                     + 2.0*(i-1==j) + 2.0*(j-1==i);
16                 mget(B,i,j) = 2.0*(i==j)
17                     + 1.0*(i-1==j) + 1.0*(j-1==i);
18             }
19
20     // Print matrices
21     print_matrix(&A);
22     print_matrix(&B);
```

Matrix Struct: main function

\$ISUHPC/lectures/lecture13/codes/matrix/main.c:

```
1  // Add/Subtract/Multiply matrices
2  matrix Csum = matrix_add(&A,&B); print_matrix(&Csum);
3  matrix Cdiff = matrix_sub(&A,&B); print_matrix(&Cdiff);
4  matrix Cprod = matrix_mult(&A,&B); print_matrix(&Cprod);
5  matrix Cdot = matrix_dot_mult(&A,&B); print_matrix(&Cdot);
6
7  // Vectors
8  vector x = new_vector(5);
9  vector y = new_vector(5);
10
11  vget(x,1) = 1.0;  vget(y,1) = 1.0;
12  vget(x,2) = 0.0;  vget(y,2) = 2.0;
13  vget(x,3) = 1.0;  vget(y,3) = 3.0;
14  vget(x,4) = 0.0;  vget(y,4) = 4.0;
15  vget(x,5) = 1.0;  vget(y,5) = 5.0;
16
17  // Print vectors
18  print_vector(&x);
19  print_vector(&y);
```

Matrix Struct: main function

\$ISUHPC/lectures/lecture13/codes/matrix/main.c:

```
1
2  // Add/Subtract/Multiply vectors
3  vector  zsum = vector_add(&x,&y); print_vector(&zsum);
4  vector  zdiff = vector_sub(&x,&y); print_vector(&zdiff);
5  double  zdot = vector_dot_mult(&x,&y); print_scalar(&zdot);
6
7  // Matrix vector multiply
8  vector Ax = matrix_vector_mult(&A,&x);
9  print_vector(&Ax);
10
11 // Linear solve via Gaussian elimination
12 vector soln = solve(&A,&y);
13 print_vector(&soln);
14 }
```

Matrix Struct: compilation & execution

```
$ make
gcc -O3 -Wall -c main.c
gcc -O3 -Wall -c matrix.c
gcc  main.o matrix.o -o main.exe
```

```
$ main.exe
```

A =

	-1.000e+00,	2.000e+00,	0.000e+00,	0.000e+00,	0.000e+00	
	2.000e+00,	-1.000e+00,	2.000e+00,	0.000e+00,	0.000e+00	
	0.000e+00,	2.000e+00,	-1.000e+00,	2.000e+00,	0.000e+00	
	0.000e+00,	0.000e+00,	2.000e+00,	-1.000e+00,	2.000e+00	
	0.000e+00,	0.000e+00,	0.000e+00,	2.000e+00,	-1.000e+00	

B =

	2.000e+00,	1.000e+00,	0.000e+00,	0.000e+00,	0.000e+00	
	1.000e+00,	2.000e+00,	1.000e+00,	0.000e+00,	0.000e+00	
	0.000e+00,	1.000e+00,	2.000e+00,	1.000e+00,	0.000e+00	
	0.000e+00,	0.000e+00,	1.000e+00,	2.000e+00,	1.000e+00	
	0.000e+00,	0.000e+00,	0.000e+00,	1.000e+00,	2.000e+00	

Matrix Struct: compilation & execution

Csum =

	1.000e+00,	3.000e+00,	0.000e+00,	0.000e+00,	0.000e+00	
	3.000e+00,	1.000e+00,	3.000e+00,	0.000e+00,	0.000e+00	
	0.000e+00,	3.000e+00,	1.000e+00,	3.000e+00,	0.000e+00	
	0.000e+00,	0.000e+00,	3.000e+00,	1.000e+00,	3.000e+00	
	0.000e+00,	0.000e+00,	0.000e+00,	3.000e+00,	1.000e+00	

Cdiff =

	-3.000e+00,	1.000e+00,	0.000e+00,	0.000e+00,	0.000e+00	
	1.000e+00,	-3.000e+00,	1.000e+00,	0.000e+00,	0.000e+00	
	0.000e+00,	1.000e+00,	-3.000e+00,	1.000e+00,	0.000e+00	
	0.000e+00,	0.000e+00,	1.000e+00,	-3.000e+00,	1.000e+00	
	0.000e+00,	0.000e+00,	0.000e+00,	1.000e+00,	-3.000e+00	

Cprod =

	0.000e+00,	3.000e+00,	2.000e+00,	0.000e+00,	0.000e+00	
	3.000e+00,	2.000e+00,	3.000e+00,	2.000e+00,	0.000e+00	
	2.000e+00,	3.000e+00,	2.000e+00,	3.000e+00,	2.000e+00	
	0.000e+00,	2.000e+00,	3.000e+00,	2.000e+00,	3.000e+00	
	0.000e+00,	0.000e+00,	2.000e+00,	3.000e+00,	0.000e+00	

Matrix Struct: compilation & execution

Cdot =

	-2.000e+00,	2.000e+00,	0.000e+00,	0.000e+00,	0.000e+00	
	2.000e+00,	-2.000e+00,	2.000e+00,	0.000e+00,	0.000e+00	
	0.000e+00,	2.000e+00,	-2.000e+00,	2.000e+00,	0.000e+00	
	0.000e+00,	0.000e+00,	2.000e+00,	-2.000e+00,	2.000e+00	
	0.000e+00,	0.000e+00,	0.000e+00,	2.000e+00,	-2.000e+00	

Matrix Struct: compilation & execution

```
x =  
| 1.000e+00, 0.000e+00, 1.000e+00, 0.000e+00, 1.000e+00 |^T
```

```
y =  
| 1.000e+00, 2.000e+00, 3.000e+00, 4.000e+00, 5.000e+00 |^T
```

```
zsum =  
| 2.000e+00, 2.000e+00, 4.000e+00, 4.000e+00, 6.000e+00 |^T
```

```
zdiff =  
| 0.000e+00, -2.000e+00, -2.000e+00, -4.000e+00, -4.000e+00 |^T
```

```
zdot =  
9.000e+00
```

```
Ax =  
| -1.000e+00, 4.000e+00, -1.000e+00, 4.000e+00, -1.000e+00 |^T
```

```
soln =  
| -1.606e+00, -3.030e-01, 2.455e+00, 3.030e+00, 1.061e+00 |^T
```

Finding Eigenvalues

Symmetric matrices

Recall: Real matrices, $A \in \mathbb{R}^{n \times n}$, that are **symmetric**:

$$A = A^T$$

only have **real eigenvalues**.

Proof: Assume A has a complex eigenvalue: λ , and the corresponding eigenvector, $\vec{v}$:

$$A\mathbf{v} = \lambda\mathbf{v}.$$

Take the complex conjugate of both sides:

$$z = x + iy \implies \bar{z} = x - iy,$$

where $i = \sqrt{-1}$:

$$A\bar{\mathbf{v}} = \bar{\lambda}\bar{\mathbf{v}}.$$

Multiply the two statements by $\bar{\mathbf{v}}^T$ and $\mathbf{v}^T$ respectively:

$$\begin{aligned}\bar{\mathbf{v}}^T A \mathbf{v} &= \lambda \bar{\mathbf{v}}^T \mathbf{v} \quad \text{and} \quad \mathbf{v}^T A \bar{\mathbf{v}} = \bar{\lambda} \mathbf{v}^T \bar{\mathbf{v}} \\ \implies \mathbf{v}^T A \bar{\mathbf{v}} &= \lambda \bar{\mathbf{v}}^T \mathbf{v} \quad \text{and} \quad \mathbf{v}^T A \bar{\mathbf{v}} = \bar{\lambda} \mathbf{v}^T \bar{\mathbf{v}} \\ 0 &= (\lambda - \bar{\lambda}) \|\mathbf{v}\|^2, \quad \text{but since } \|\mathbf{v}\| > 0 \\ \implies \lambda &= \bar{\lambda} \implies \lambda \text{ is real.}\end{aligned}$$

Rayleigh Quotient

Problem: Given a vector $\mathbf{x} \in \mathbb{R}^n$ and a real symmetric matrix $A \in \mathbb{R}^{n \times n}$, find a real number $\hat{\alpha} \in \mathbb{R}$ such that

$$\|A\mathbf{x} - \hat{\alpha}\mathbf{x}\| = \min_{\alpha \in \mathbb{R}} \|A\mathbf{x} - \alpha\mathbf{x}\|.$$

Rayleigh Quotient

Problem: Given a vector $\mathbf{x} \in \mathbb{R}^n$ and a real symmetric matrix $A \in \mathbb{R}^{n \times n}$, find a real number $\hat{\alpha} \in \mathbb{R}$ such that

$$\|A\mathbf{x} - \hat{\alpha}\mathbf{x}\| = \min_{\alpha \in \mathbb{R}} \|A\mathbf{x} - \alpha\mathbf{x}\|.$$

Solution:

$$\begin{aligned}\|A\mathbf{x} - \alpha\mathbf{x}\|^2 &= (A\mathbf{x} - \alpha\mathbf{x})^T (A\mathbf{x} - \alpha\mathbf{x}) = (\mathbf{x}^T A - \alpha\mathbf{x}^T) (A\mathbf{x} - \alpha\mathbf{x}) \\ &= \mathbf{x}^T A^2 \mathbf{x} - 2\alpha\mathbf{x}^T A \mathbf{x} + \alpha^2 \mathbf{x}^T \mathbf{x}\end{aligned}$$

(concave up quadratic function, unique global min where $d/d\alpha$ is zero)

$$\frac{d}{d\alpha} (\mathbf{x}^T A^2 \mathbf{x} - 2\alpha\mathbf{x}^T A \mathbf{x} + \alpha^2 \mathbf{x}^T \mathbf{x}) = 0 \quad \implies \quad 2 (\alpha\mathbf{x}^T \mathbf{x} - \mathbf{x}^T A \mathbf{x}) = 0$$

$$\therefore \quad \hat{\alpha} = R_A(\mathbf{x}) = \frac{\mathbf{x}^T A \mathbf{x}}{\mathbf{x}^T \mathbf{x}} \quad (\text{Rayleigh Quotient})$$

Rayleigh Quotient

Problem: Given a vector $\mathbf{x} \in \mathbb{R}^n$ and a real symmetric matrix $A \in \mathbb{R}^{n \times n}$, find a real number $\hat{\alpha} \in \mathbb{R}$ such that

$$\|A\mathbf{x} - \hat{\alpha}\mathbf{x}\| = \min_{\alpha \in \mathbb{R}} \|A\mathbf{x} - \alpha\mathbf{x}\|.$$

Solution:

$$\begin{aligned}\|A\mathbf{x} - \alpha\mathbf{x}\|^2 &= (A\mathbf{x} - \alpha\mathbf{x})^T (A\mathbf{x} - \alpha\mathbf{x}) = (\mathbf{x}^T A - \alpha\mathbf{x}^T) (A\mathbf{x} - \alpha\mathbf{x}) \\ &= \mathbf{x}^T A^2 \mathbf{x} - 2\alpha\mathbf{x}^T A\mathbf{x} + \alpha^2 \mathbf{x}^T \mathbf{x}\end{aligned}$$

(concave up quadratic function, unique global min where $d/d\alpha$ is zero)

$$\frac{d}{d\alpha} (\mathbf{x}^T A^2 \mathbf{x} - 2\alpha\mathbf{x}^T A\mathbf{x} + \alpha^2 \mathbf{x}^T \mathbf{x}) = 0 \quad \implies \quad 2(\alpha\mathbf{x}^T \mathbf{x} - \mathbf{x}^T A\mathbf{x}) = 0$$

$$\therefore \quad \hat{\alpha} = R_A(\mathbf{x}) = \frac{\mathbf{x}^T A\mathbf{x}}{\mathbf{x}^T \mathbf{x}} \quad (\text{Rayleigh Quotient})$$

If $\mathbf{v}$ is an eigenvector of A , then

$$R_A(\mathbf{v}) = \frac{\mathbf{v}^T A\mathbf{v}}{\mathbf{v}^T \mathbf{v}} = \frac{\mathbf{v}^T \lambda \mathbf{v}}{\mathbf{v}^T \mathbf{v}} = \lambda \left(\frac{\mathbf{v}^T \mathbf{v}}{\mathbf{v}^T \mathbf{v}} \right) = \lambda.$$

Estimating the largest eigenvalue

Assume:

$$|\lambda_1| > |\lambda_2| > |\lambda_3| > \cdots > |\lambda_n|$$

Estimating the largest eigenvalue

Assume:

$$|\lambda_1| > |\lambda_2| > |\lambda_3| > \cdots > |\lambda_n|$$

Guess and eigenvector:

$$\mathbf{w}^{(0)} = \sum_{j=1}^n \alpha_j \mathbf{v}_j$$

where $\mathbf{v}_j$ for $j = 1, 2, \dots, n$ are the n linearly independent eigenvectors of A .

Estimating the largest eigenvalue

Assume:

$$|\lambda_1| > |\lambda_2| > |\lambda_3| > \dots > |\lambda_n|$$

Guess and eigenvector:

$$\mathbf{w}^{(0)} = \sum_{j=1}^n \alpha_j \mathbf{v}_j$$

where $\mathbf{v}_j$ for $j = 1, 2, \dots, n$ are the n linearly independent eigenvectors of A .

How do we improve this guess?

$$\mathbf{w}^{(1)} = A\mathbf{w}^{(0)} = \sum_{j=1}^n \alpha_j A\mathbf{v}_j = \sum_{j=1}^n \alpha_j \lambda_j \mathbf{v}_j$$

$$\mathbf{w}^{(2)} = A\mathbf{w}^{(1)} = A^2\mathbf{w}^{(0)} = \sum_{j=1}^n \alpha_j A^2\mathbf{v}_j = \sum_{j=1}^n \alpha_j \lambda_j^2 \mathbf{v}_j$$

$$\mathbf{w}^{(k)} = A\mathbf{w}^{(k-1)} = A^k\mathbf{w}^{(0)} = \sum_{j=1}^n \alpha_j A^k\mathbf{v}_j = \sum_{j=1}^n \alpha_j \lambda_j^k \mathbf{v}_j \approx \alpha_1 \lambda_1^k \mathbf{v}_1$$

$$\therefore \lambda_1 \approx R_A(\mathbf{w}^{(k)})$$

Power Iteration Algorithm

INPUT: $\mathbf{v}^{(0)} \in \mathbb{R}^n$, TOL, MaxIters, $A \in \mathbb{R}^{n \times n}$ (real & symmetric)

OUTPUT: $\lambda^{(k)}$ (approximation to λ_1 after k iterations)

$$\mathbf{v}^{(0)} = \frac{\mathbf{v}^{(0)}}{\|\mathbf{v}^{(0)}\|} \quad \text{and} \quad \lambda^{(0)} = \left[\mathbf{v}^{(0)} \right]^T A \mathbf{v}^{(0)}$$

mstop = 0 and $k = 0$

while mstop==0 **do**

$k = k + 1$

$$\mathbf{w} = A \mathbf{v}^{(k-1)}, \quad \mathbf{v}^{(k)} = \frac{\mathbf{w}}{\|\mathbf{w}\|}, \quad \text{and} \quad \lambda^{(k)} = \left[\mathbf{v}^{(k)} \right]^T A \mathbf{v}^{(k)}$$

if $|\lambda^{(k)} - \lambda^{(k-1)}| < \text{TOL}$ **or** $k == \text{MaxIters}$ **then**

mstop = 1

end if

end while